SC3098 GROUP DESIGN PROJECT:

GETTING STARTED WITH CHATBOT DEVELOPMENT

by Ong Chin Ann | Lecturer
College of Computing & Data Science



NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

CDEFG
Chatbot Developers and Enthusiasts Focus Group

Streamlit LangChain

OVERVIEW

This materials provide you the necessary information and guidance to take full-control on application or chatbot that you wish to develop:

1. Learn the Basics of Chatbot Development: Discover how to create a simple yet effective chatbot using the powerful and intuitive Streamlit framework.
2. Familiarized with the concept of RAG for chatbot development

By the end of this session, you will have developed a custom chatbot and successfully deployed it locally for experiment and future development. Whether you're a beginner taking your first steps in development or an experienced developer looking to expand your skillset, this session will equip you with the tools and knowledge to build, manage, and secure your own chatbot application.

AGENDA

- Introduction to Context Aware Chatbot Development
  - Chatbot Architecture
  - Retrieval Augmented Generation (RAG)
  - Prompt Engineering
- Developing LLM-Powered RAG Chatbot with Streamlit
- NTU Chatbot & LLM-Powered System Development Community

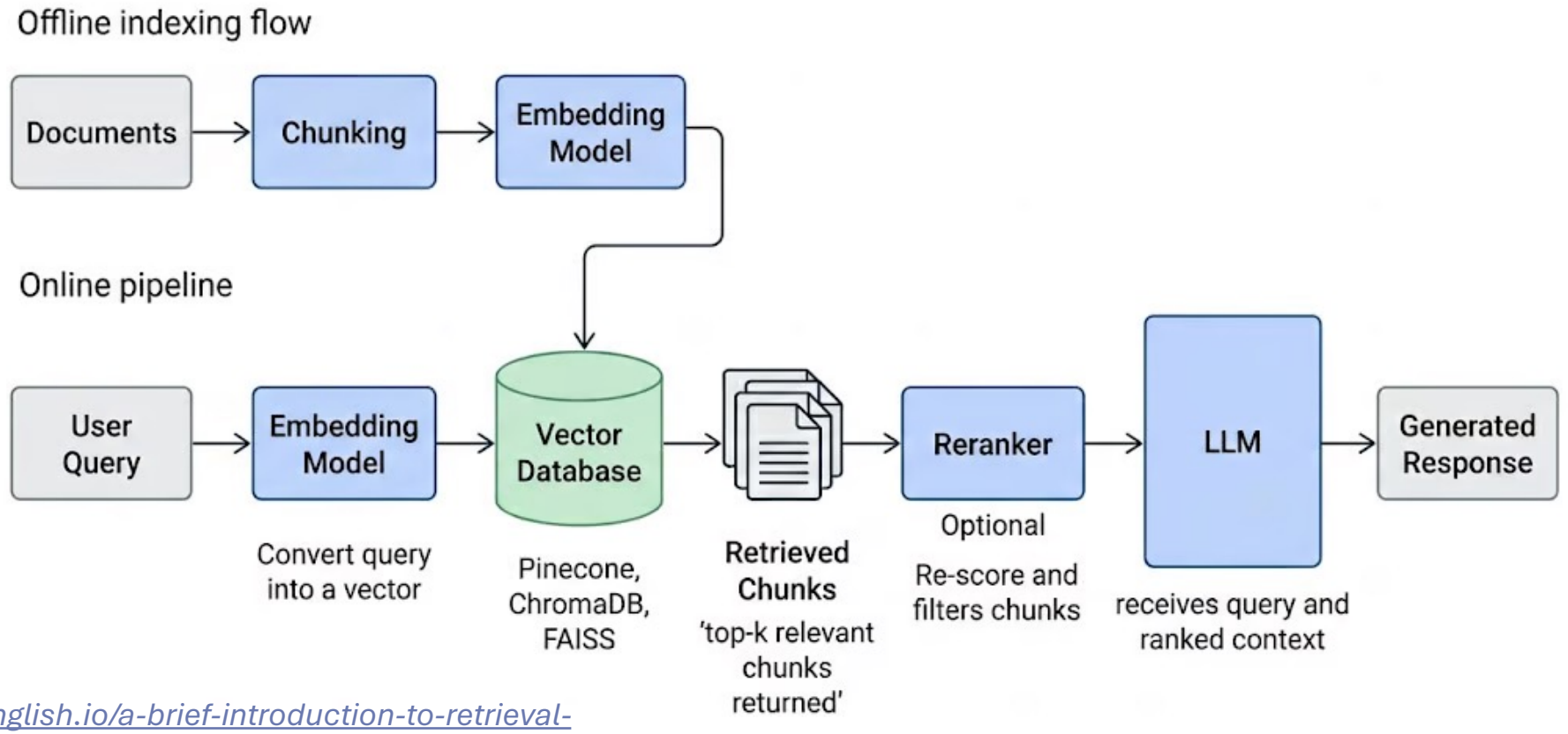
FASTEN YOUR SEATBELT

- We are covering a few technical topics in ~50 minutes.
- There might be "*turbulence*" throughout the journey.
- Ensure you fasten your seatbelt and full pay attention so you will not be thrown away!!!

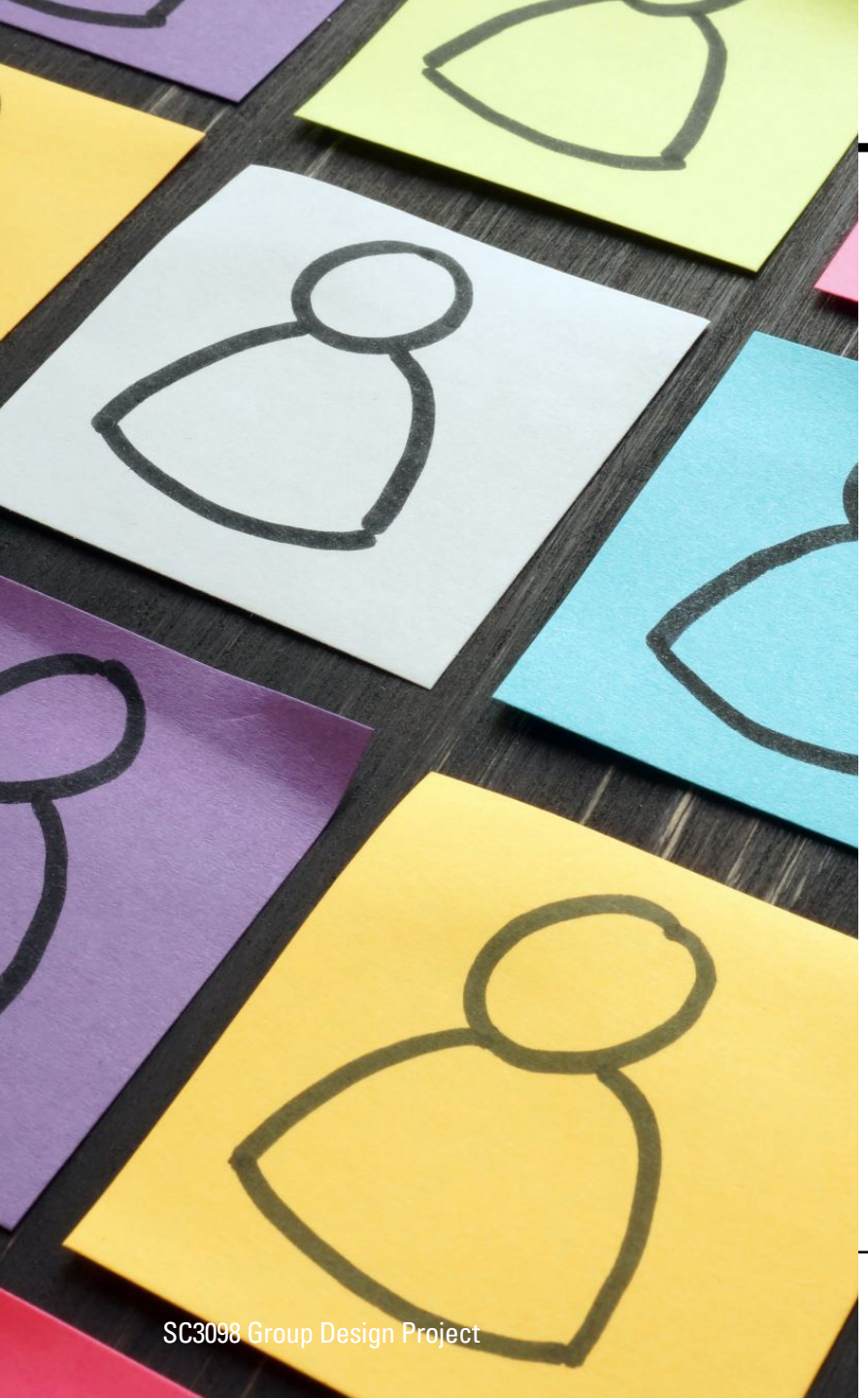


Image Source: giphy

HOW DOES RAG BASED CHATBOT WORKS?



Source: *https://ai.plainenglish.io/a-brief-introduction-to-retrieval-augmented-generation-rag-b7eb70982891*



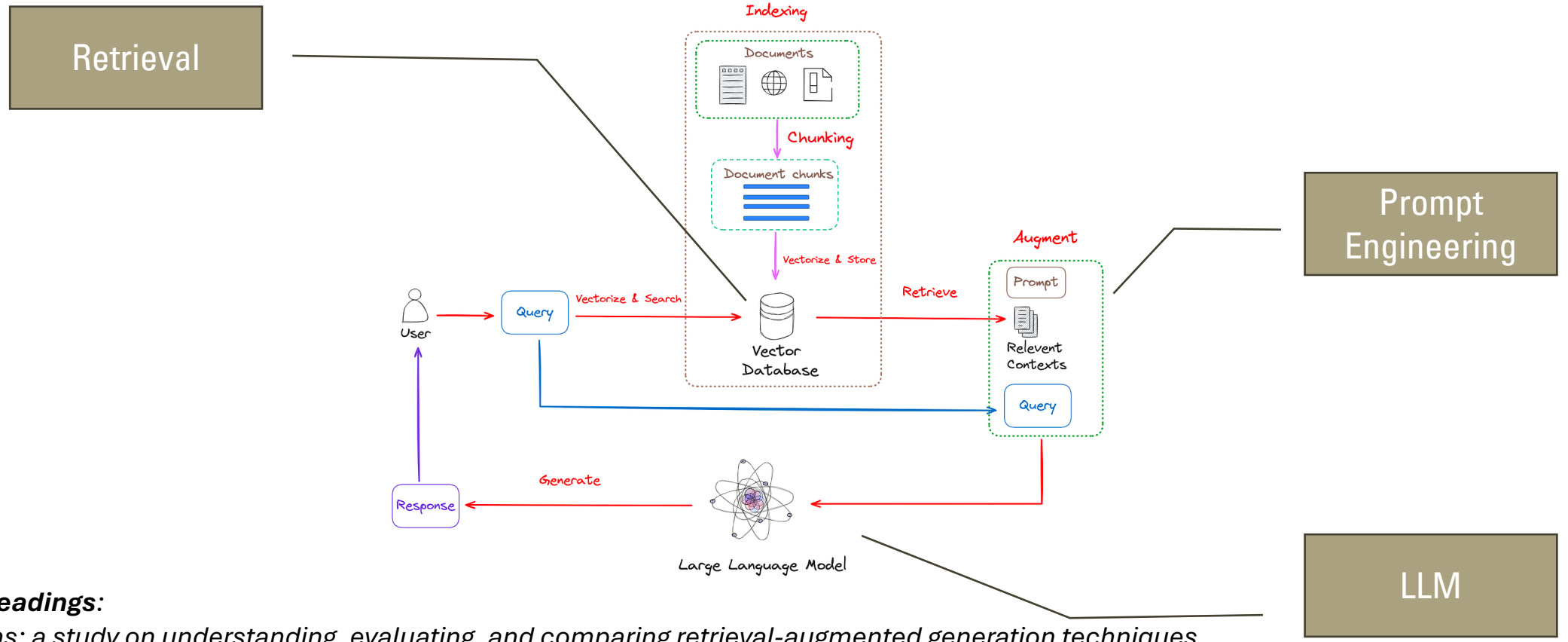
BASIC PRINCIPLES OF PROMPT ENGINEERING (SYSTEM PROMPT)

Write a clear and specific instructions:

- Define the chatbot role/persona
- the task
- context/data
- conditions/constraint
- output format
- example (few-shot)

https://www.deeplearning.ai/courses/chatgpt-prompt-eng

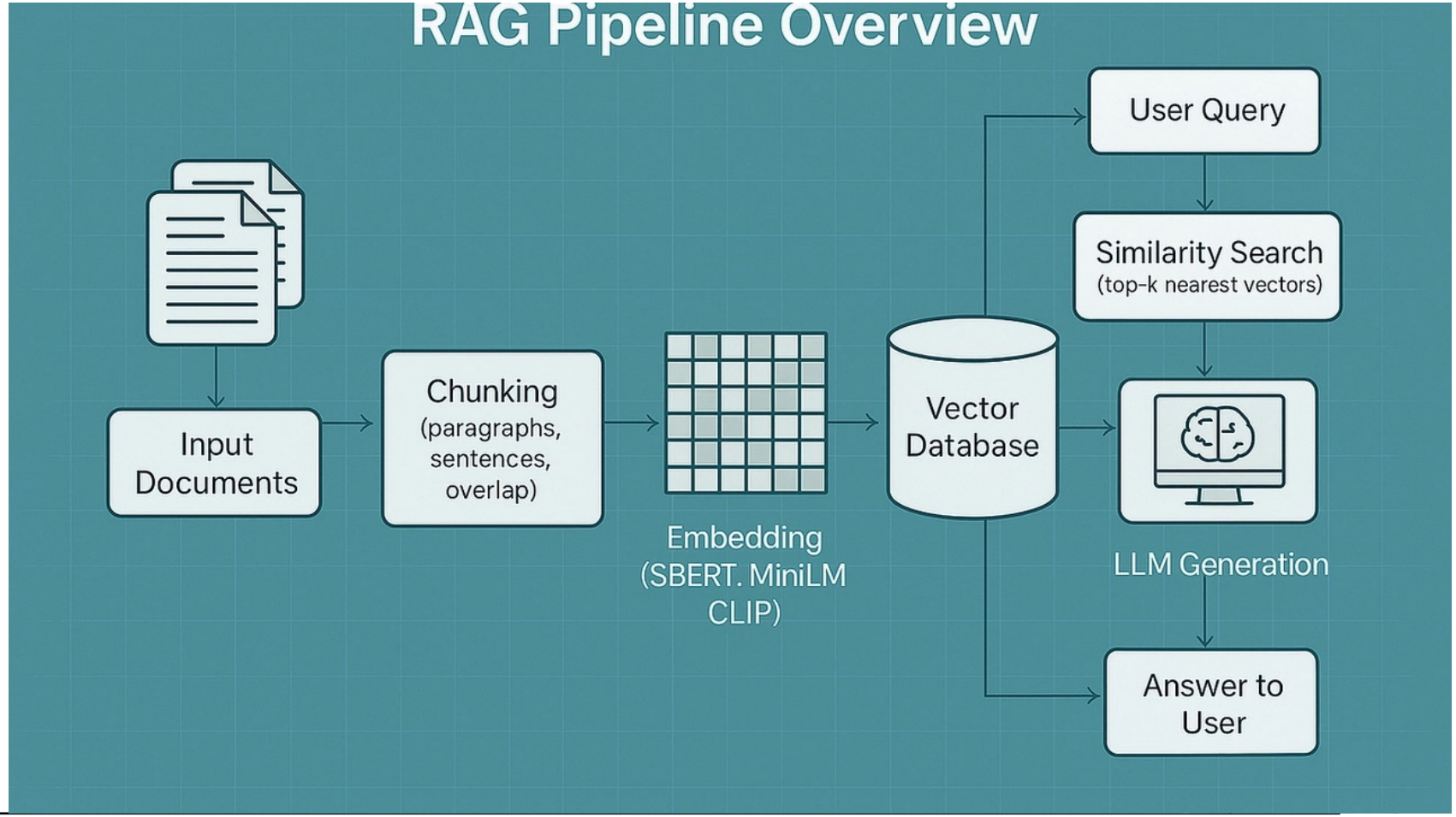
HOW DOES RAG BASED CHATBOT WORKS?



More Readings:

RAGLens: a study on understanding, evaluating, and comparing retrieval-augmented generation techniques (https://hdl.handle.net/10356/214393)

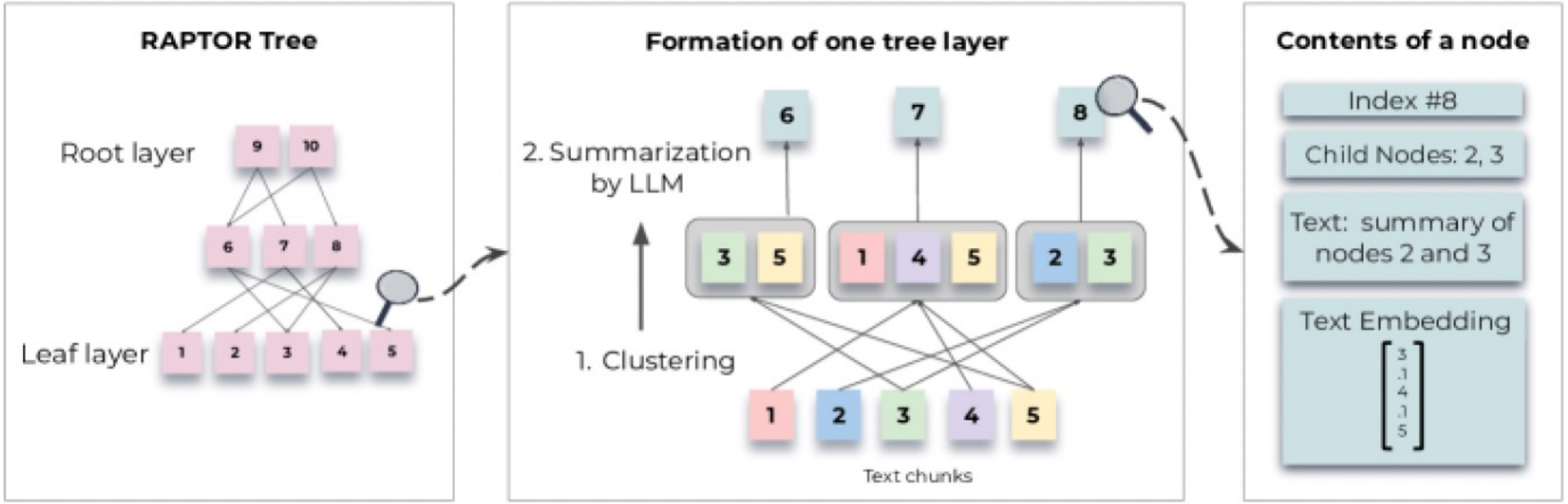
DOCUMENT LOADING, SPLIT & CHUNKING, EMBEDDINGS



Sources:
https://dev.to/derrickryangiggs/rag-pipeline-deep-dive-ingestion-chunking-embedding-and-vector-search-2877
https://www.pinecone.io/learn/chunking-strategies/

OTHER ADVANCE RAG TECHNIQUES

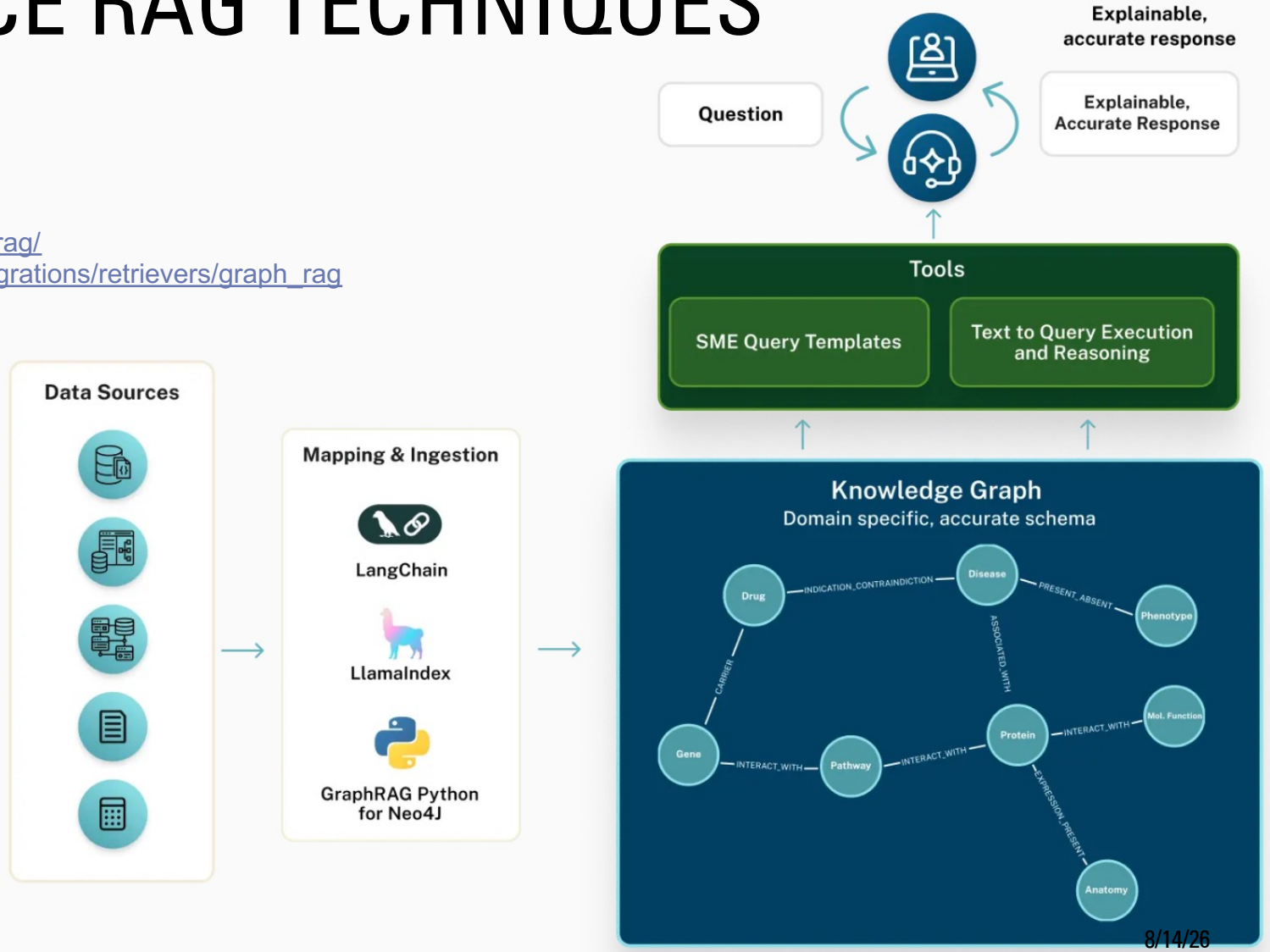
- Recursive Abstractive Processing for Tree-Organized Retrieval (RAPTOR)



Sarthi, P., Abdullah, S., Tuli, A., Khanna, S., Goldie, A., & Manning, C. (2024, May). Raptor: Recursive abstractive processing for tree-organized retrieval. In *International Conference on Learning Representations* (Vol. 2024, pp. 32628-32649). https://arxiv.org/abs/2401.18059

OTHER ADVANCE RAG TECHNIQUES

- Graph RAG
  - https://neo4j.com/use-cases/ai-systems/
  - https://neo4j.com/blog/genai/what-is-graphrag/
  - https://docs.langchain.com/oss/python/integrations/retrievers/graph_rag



OTHER ADVANCE RETRIEVAL TECHNIQUES

- And Many More…
  - HyDE (Hypothetical Document Embeddings)
  - Parent-Child Retriever
  - Hypothetical Question Retriever
  - Query Rewriting / Expansion
  - …

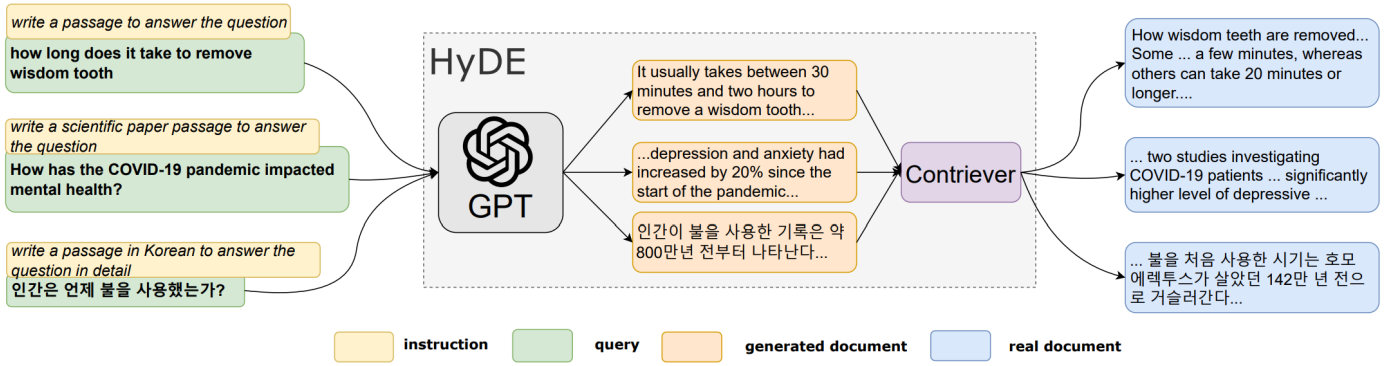
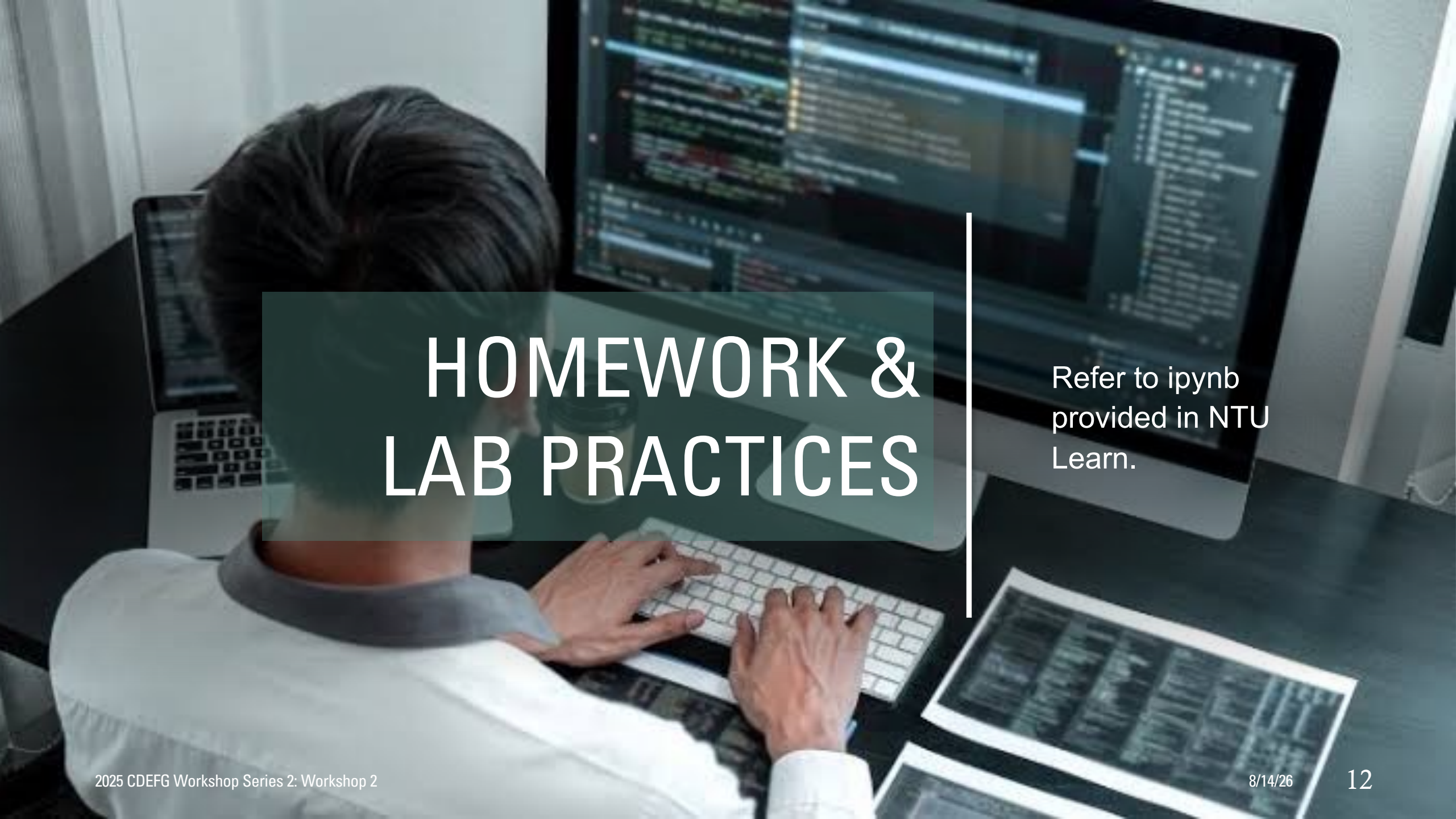


Figure 1: An illustration of the HyDE model. Document snippets are shown. HyDE serves all types of queries without changing the underlying InstructGPT and Contriever/mContriever models.

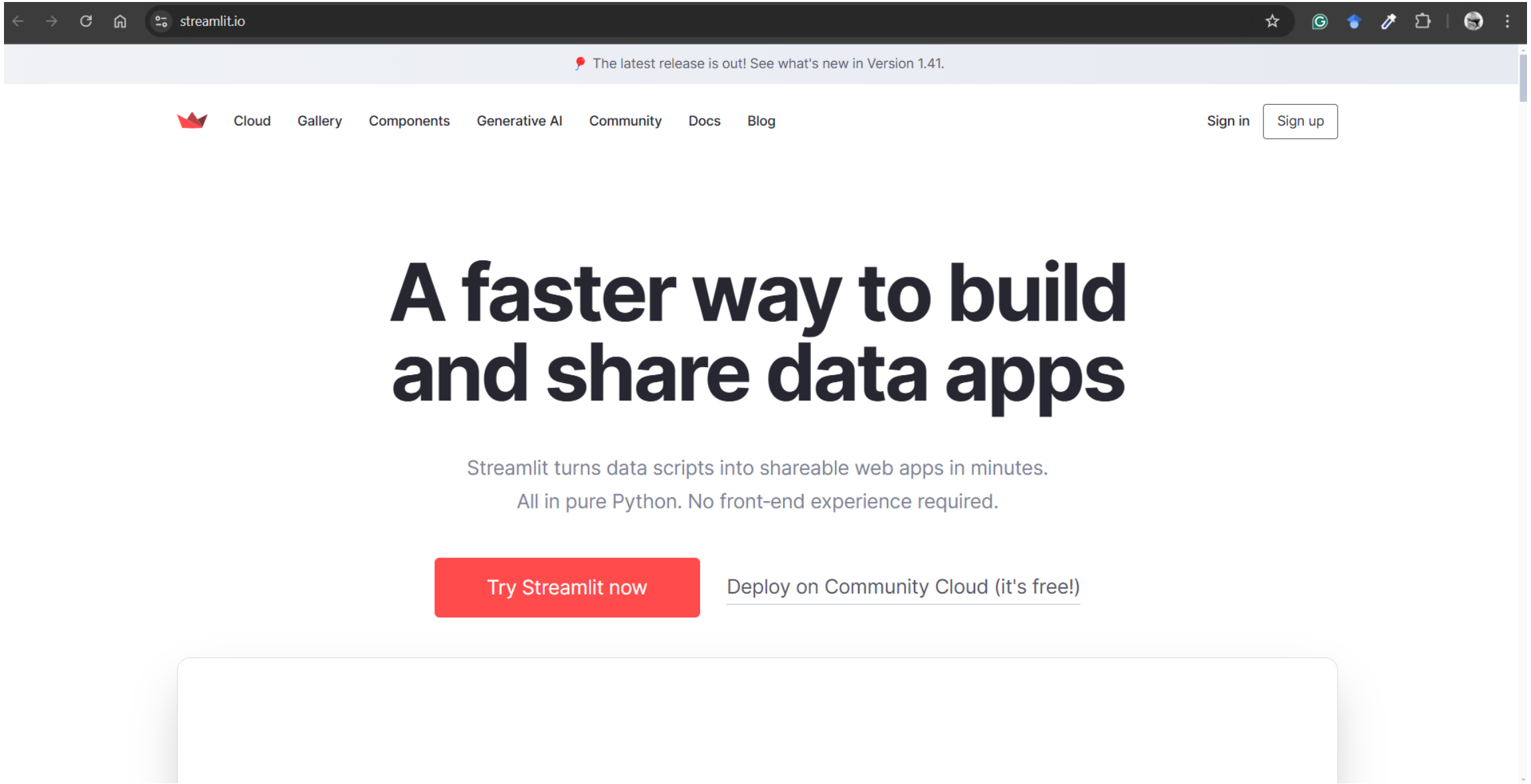
https://graphrag.com/reference/graphrag/basic-retriever/
https://docs.haystack.deepset.ai/docs/hypothetical-document-embeddings-hyde



HOMEWORK & LAB PRACTICES

Refer to ipynb provided in NTU Learn.

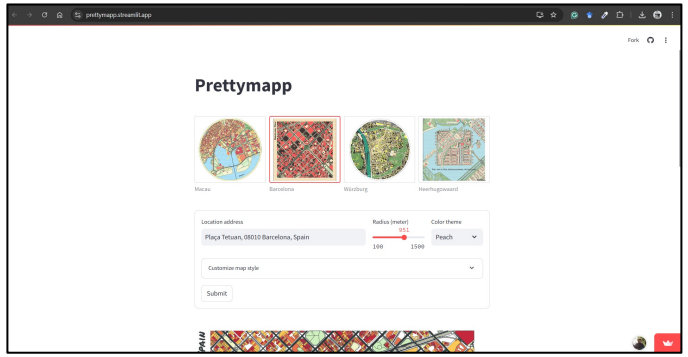
EXPLORING STREAMLIT UI & CHAT AGENT DEVELOPMENT


streamlit.io
The latest release is out! See what's new in Version 1.41.
Cloud
Gallery
Components
Generative AI
Community
Docs
Blog
Sign in
Sign up
A faster way to build and share data apps
Streamlit turns data scripts into shareable web apps in minutes.
All in pure Python. No front-end experience required.
Try Streamlit now
Deploy on Community Cloud (it's free!)

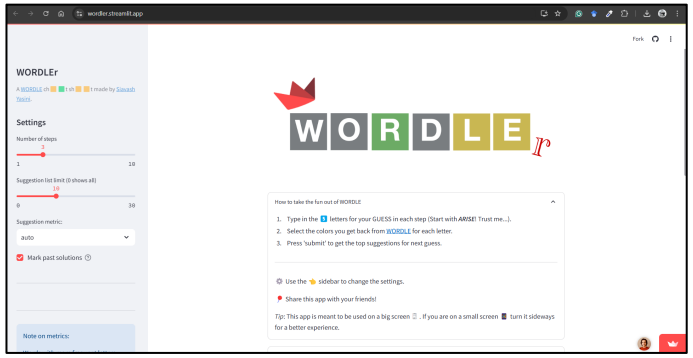


My Favorite Text Editor

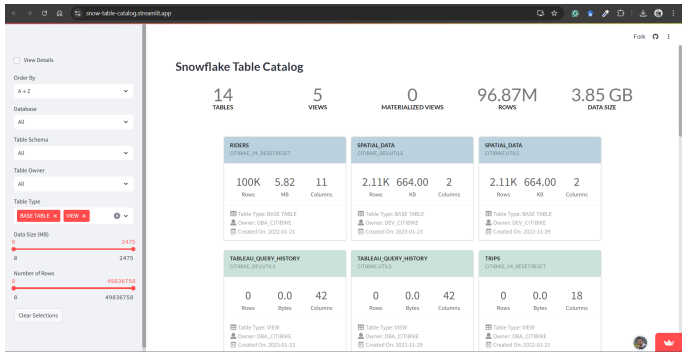
WHAT CAN YOU DO WITH STREAMLIT?



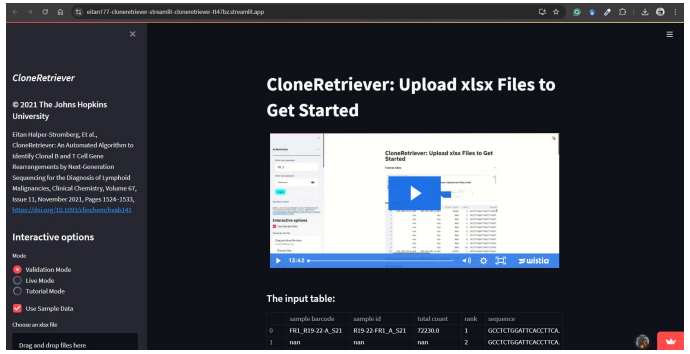
Custom Map Generator



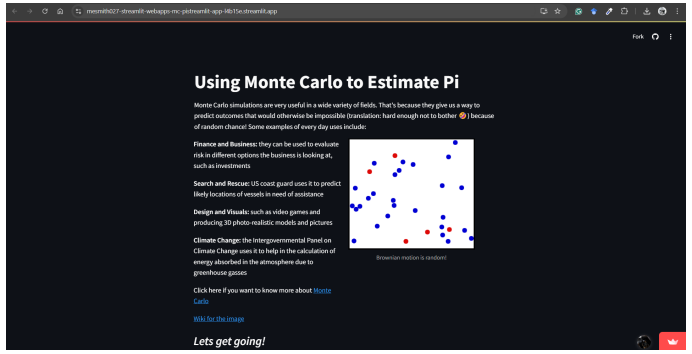
Word Games



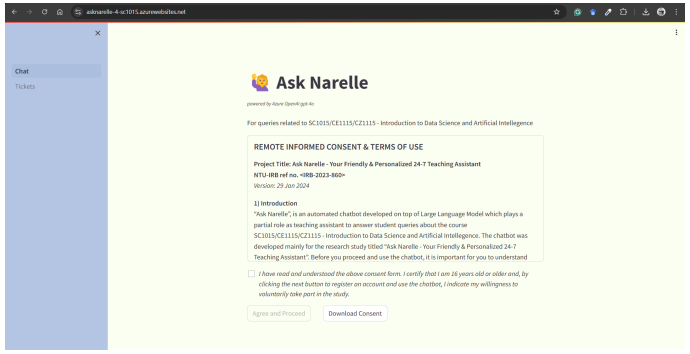
Dashboard



Data Science Project

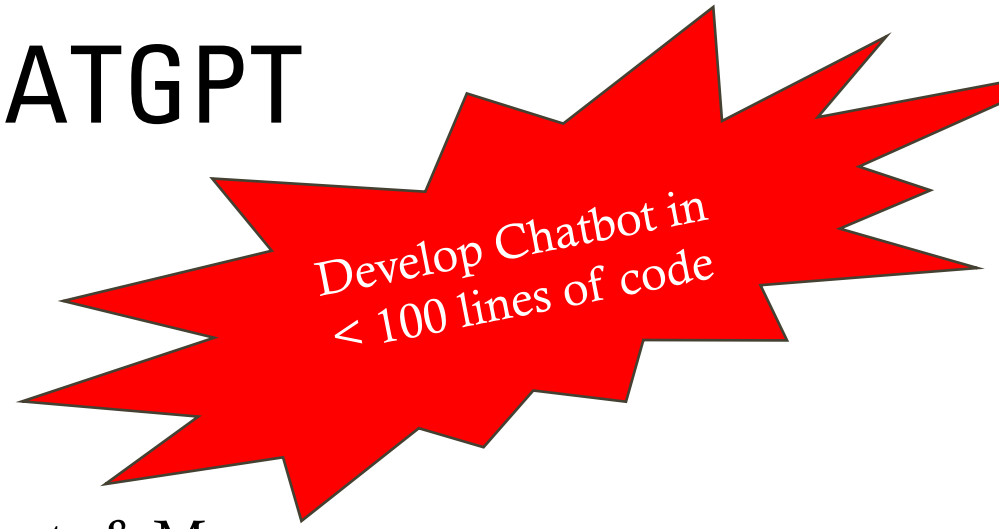


Simulator



Chatbot

DEVELOPING <YOURNAME>-CHATGPT



- **Step 1:** Setting up working environment & Installation
- **Step 2:** Create a simple page and run on local machine
- **Step 3:** Incorporating conversational history with chat elements & Messages
- **Step 4**: Incorporating LLM & knowledge base (Session 1 - Workshop 2)
  - Dynamic Knowledge Base Construction will be covered on Friday (Session 5 – Workshop 1)
- **Step 5:** Test out your chatbot and grow further!!! **(Homework)**

STEP 1: SETTING UP WORKING ENVIRONMENT & INSTALLATION

- Activate your previous python environment and go to **“workshop 2”** directory
- Pip install the following packages:
  - `pip install python-dotenv streamlit langchain langchain-openai langchain-core langchain-community openai faiss-cpu`
- In CLI (command line/terminal) execute to test the setup/installation:
  - `streamlit hello`

UI COMPONENTS, WIDGETS, CONTROLS

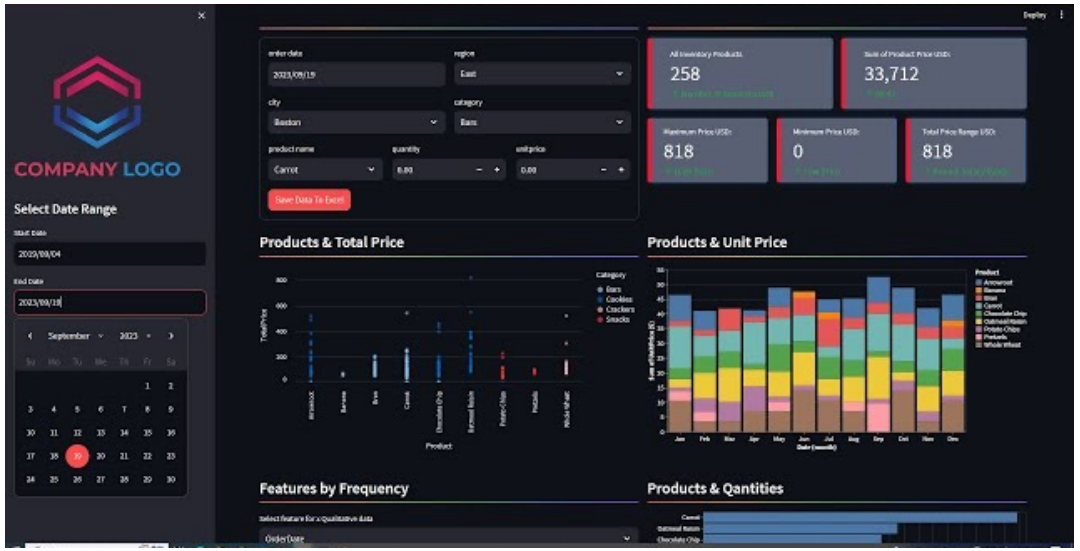


Image Source: https://youtu.be/rJzBNm0rb0g

- Streamlit comes with many widgets / UI Components
  - https://docs.streamlit.io/get-started/fundamentals/main-concepts#widgets

STEP 2: CREATE A SIMPLE PAGE WITH TEXT ELEMENT AND RUN ON LOCAL MACHINE

- Create an empty file named “main.py” in “workshop 2” directory.
- Type the following code and **save**:

main.py > ...

```python
import streamlit as st

st.set_page_config(page_title= "ChatBot", page_icon=":books:")
st.title(":books: AskNarelle")
st.header("This is a header")
st.markdown("Hello World :sunglasses:")
```

Extracted from Tan Wei Yin (CCDS), workshop handout

- In CLI execute:
  - **`streamlit run main.py`**

Note: :books: and :sunglasses: are the icons for the Streamlit app.
https://streamlit-emoji-shortcodes-streamlit-app-gwckff.streamlit.app/

STEP 2: CREATE A SIMPLE PAGE WITH SIDEBAR AND RADIO BUTTONS

main.py > ...

```python
import streamlit as st

st.set_page_config(page_title= "ChatBot", page_icon=":books:")
st.title(":books: AskNarelle")

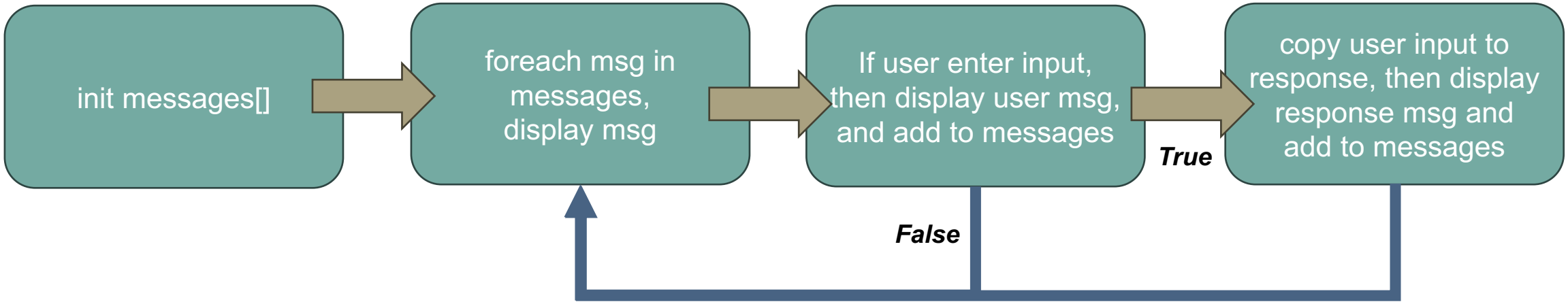
with st.sidebar:
    st.header("Profile")
    gender = st.radio(
        label = "Gender",
        options = ("Male", "Female")
    )
    if gender == "Male":
        st.header("Welcome Guys :boy:")
    else:
        st.header("Welcome Girls :girl:")
```

- Extend the following code, **save**, and click "rerun" on the webpage.

Extracted from Tan Wei Yin (CCDS), workshop handout

STEP 3: INCORPORATING CONVERSATIONAL HISTORY WITH CHAT_MESSAGE AND CHAT_INPUT

- Logical flow of streamlit app capturing user input, iterate, and display messages



STEP 3A: INCORPORATING CONVERSATIONAL HISTORY WITH CHAT_MESSAGE AND CHAT_INPUT

- **Right above sidebar**, extend the following code, **save**, and click "rerun" on the webpage.
- Initialize messages (chat history)
- Looping through messages and display message
- Acquiring user input and display user input

```
# Initialize session state
if "messages" not in st.session_state:
    st.session_state['messages'] = []
```

```
# Display chat messages from
for message in st.session_state.messages:
    with st.chat_message("user"):
        st.markdown(message)


# Widget for user input, noted on the equal compare sign :=
if prompt := st.chat_input("Say something"):
    with st.chat_message("user"):
        st.markdown(prompt)
        st.session_state.messages.append(prompt)
```

Extracted partially from Tan Wei Yin (CCDS), workshop handout

Session State

Session State provides a dictionary-like interface where you can save information that is preserved between script reruns. Use `st.session_state` with key or attribute notation to store and recall values. For example, `st.session_state["my_key"]` or `st.session_state.my_key`. Remember that widgets handle their statefulness all by themselves, so you won't always need to use Session State!

What is a session?

A session is a single instance of viewing an app. If you view an app from two different tabs in your browser, each tab will have its own session. So each viewer of an app will have a Session State tied to their specific view. Streamlit maintains this session as the user interacts with the app. If the user refreshes their browser page or reloads the URL to the app, their Session State resets and they begin again with a new session.

https://docs.streamlit.io/get-started/fundamentals/advanced-concepts#session-state

STEP 3B: INCORPORATING CONVERSATIONAL HISTORY WITH CHAT_MESSAGE AND CHAT_INPUT

- Now we try to echo what we type to simulate chatbot response.
- Copy user input to response
- Display response message

```python
# Display chat messages from
for message in st.session_state.messages:
    with st.chat_message("user"):
        st.markdown(message)


# Widget for user input, noted on the equal compare sign :=
if prompt := st.chat_input("Say something"):
    with st.chat_message("user"):
        st.markdown(prompt)
        st.session_state.messages.append(prompt)

    # Echo User Response / We will replace this with LLM Engine later
    response = f"You Typed: {prompt}"

    # Display assistant message in chat message container
    with st.chat_message("assistant"):
        st.markdown(response)
        st.session_state.messages.append(response)
```

Chat Avatar Incorrect

Extracted partially from Tan Wei Yin (CCDS), workshop handout

STEP 3C: INCORPORATING CONVERSATIONAL HISTORY WITH CHAT_MESSAGE AND CHAT_INPUT

- Instead of storing plain string, we will now store dict object for each message.
- **Update the code in yellow box**

Save file, and rerun again

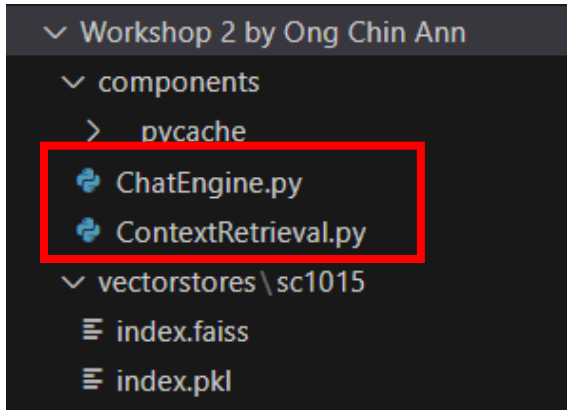
```
# Display chat messages from
for message in st.session_state.messages:
    with st.chat_message(message['role']):
        st.markdown(message['content'])

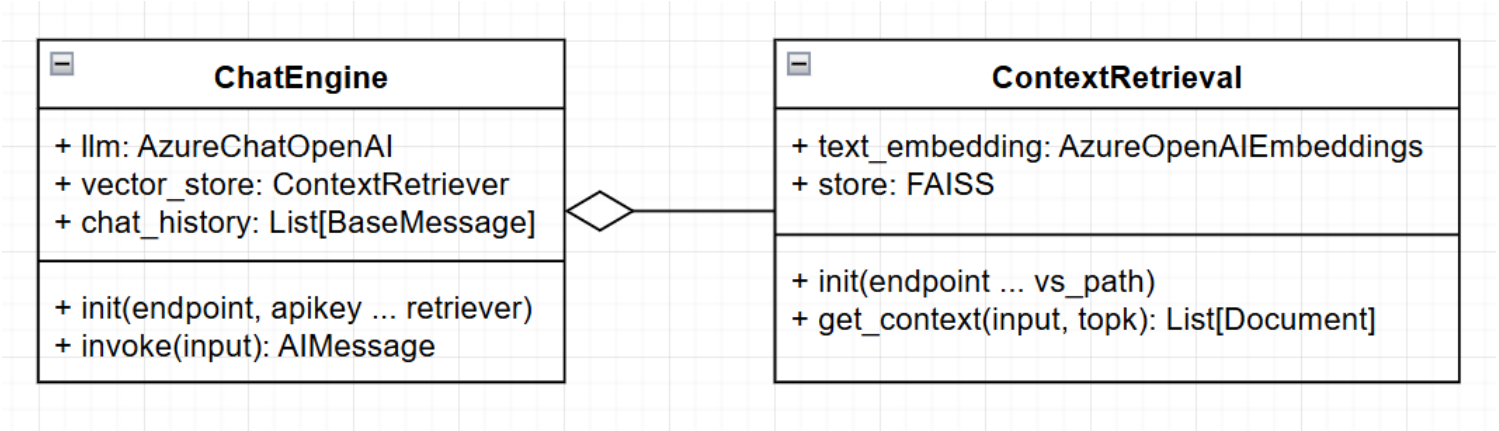

# Widget for user input, noted on the equal compare sign :=
if prompt := st.chat_input("Say something"):
    with st.chat_message("user"):
        st.markdown(prompt)
        st.session_state.messages.append({"role":"user", "content":prompt})

    # Echo User Response / We will replace this with LLM Engine later
    response = f"You Typed: {prompt}"

    # Display assistant message in chat message container
    with st.chat_message("assistant"):
        st.markdown(response)
        st.session_state.messages.append({"role":"assistant", "content":response})
```

STEP 4: INCORPORATING LLM & KNOWLEDGE BASE (CUSTOM DEFINED CLASS - OOP)





- **ChatEngine**: an object that interact with LLM, keep chat history, able to consult vector database via a context retrieval. (*Custom Chat Agent*)
- **ContextRetrieval**: an object that interact with FAISS vector store, able perform document search using similarity search technique.

STEP 4A: INCORPORATING LLM & KNOWLEDGE BASE

- **ChatEngine**: an object that interact with LLM, keep chat history, able to consult vector database via a context retrieval.
- **ContextRetrieval**: an object that interact with FAISS vector store, able perform document search using similarity search technique.

```python
# Initialize session state
if "messages" not in st.session_state:
    st.session_state['messages'] = []

    # Initialized Context Retriever
    st.session_state.retriever = ContextRetriever(
        endpoint=os.environ['AZURE_OPENAI_ENDPOINT'],
        apikey=os.environ['AZURE_OPENAI_APIKEY'],
        deployment_name=os.environ['AZURE_TEXT_EMBEDDING'],
        model_name=os.environ['AZURE_TEXT_EMBEDDING_MODEL'],
        vs_path="vectorstores/sc1015"
    )

    # Initialized LLM
    st.session_state.llm = ChatEngine(
        endpoint=os.environ['AZURE_OPENAI_ENDPOINT'],
        apikey=os.environ['AZURE_OPENAI_APIKEY'],
        deployment_name=os.environ['AZURE_OPENAI_DEPLOYMENT_NAME'],
        model_name=os.environ['AZURE_OPENAI_MODEL_NAME'],
        api_version=os.environ['AZURE_OPENAI_API_VERSION'],
        retriever=st.session_state.retriever
    )
```

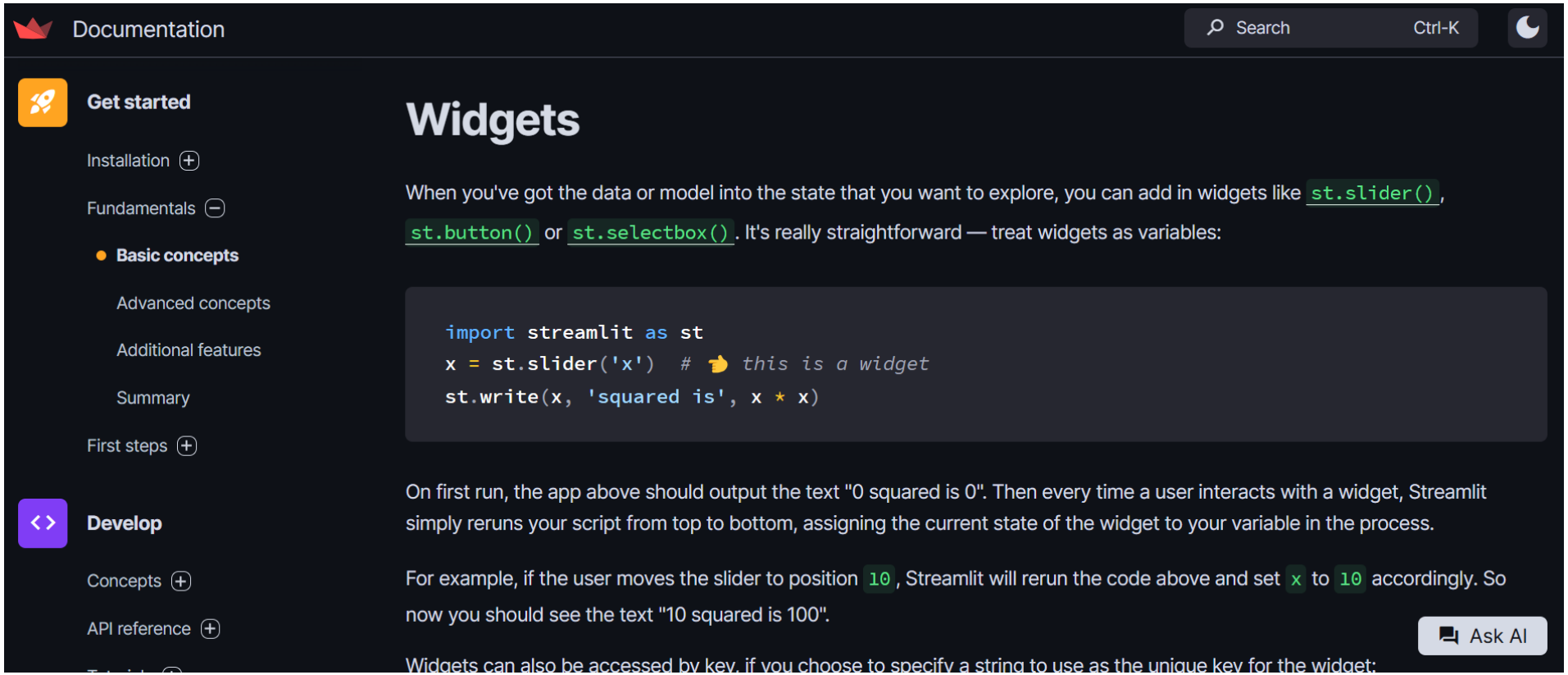
```python
# Widget for user input, noted on the equal compare sign :=
if prompt := st.chat_input("Say something"):
    with st.chat_message("user"):
        st.markdown(prompt)
        st.session_state.messages.append({"role":"user", "content":prompt})

    # Echo User Response / We will replace this with LLM Engine later
    # response = f"You Typed: {prompt}"
    response = st.session_state.llm.invoke(prompt).content
```

MISSION ACCOMPLISHED

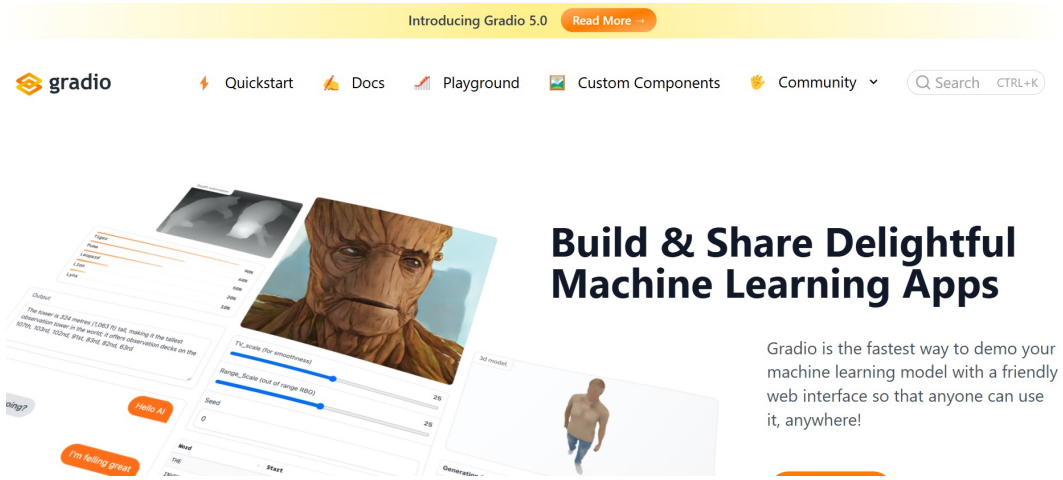
Task 5: *Test out your chatbot and grow further!!!*

BEYOND WHAT WE COVER

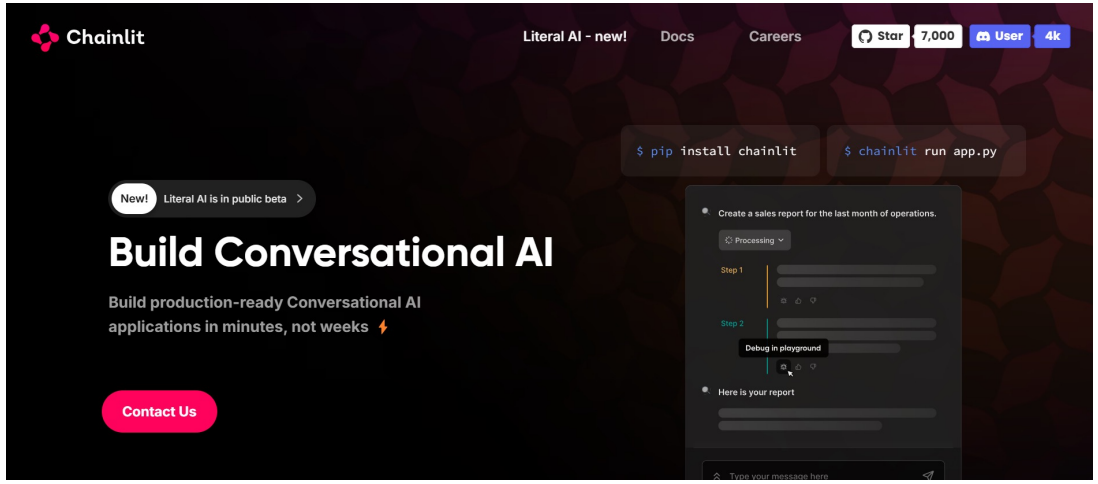


https://docs.streamlit.io/get-started/fundamentals/main-concepts

OTHER SIMILAR FRAMEWORKS



https://www.gradio.app/



https://chainlit.io/

Want to learn more?

CHATBOT DEVELOPER & ENTHUSIAST FOCUS GROUP (CDEFG)



BY STUDENTS, FOR STUDENTS

NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

ABOUT THE FOCUS GROUP

Serve as a Platform for

Likeminded people (student/staff)

- Learning
- Sharing (Work showcase)
- Technological Exploration (Workshop)

Experience Students

- Prepare for FYP/URECA
- Work showcase (portfolio) – self assessment
- Look for suitable team members
- Seek for projects / opportunity

Profs/Employer

- Look for Project Assistant/Dev
- Look for Intern
- Organize related events



COMMUNICATION & FILE SHARING

For ease of communication and file sharing, we will be using MS Team under the [Chatbot Developers & Enthusiasts Focus Group (CDEFG)]

SCAN ME

PAST CDEFG EVENTS



- https://www.linkedin.com/posts/ugcPost-7199239766213627904-wcU8/
- https://www.linkedin.com/posts/ongca_over-the-past-six-weeks-i-had-the-opportunity-ugcPost-7296179376134529025-agws
- https://www.linkedin.com/posts/ongca_fypstudents-leadership-cdefg-ugcPost-7446807908597792768-LBF5/